

Sistemi Operativi

Simulazione del 4 giugno 2026 – Compito A



Indicazioni generali

- Il tempo a disposizione per la prova è di 2 ore inclusi gli esercizi e le domande teoriche.
- Ciascuna risposta corretta della parte teorica contrassegnata vale 0.5 punti, -0.25 in caso di errore.
- Ogni domanda di teoria ha tra una e tre risposte giuste.
- Ciascun esercizio di laboratorio vale fino a 10 punti.
- Per gli esercizi è disponibile un validatore da scaricare da moodle. Il validatore può essere usato liberamente.
- Il punteggio effettivo viene attribuito sulla base del risultato dei validatori e della qualità del codice prodotto.
- È vietato l'uso di strumenti di generazione automatica del codice e la comunicazione con l'esterno.

1. Quali affermazioni distinguono correttamente indirizzi logici e fisici?

1. Un indirizzo fisico è sempre generato direttamente dal codice sorgente C
2. Un indirizzo logico è generato dalla CPU ed è anche detto indirizzo virtuale
3. Un indirizzo fisico è l'indirizzo visto dall'unità di memoria
4. Con binding a execution time, indirizzi logici e fisici devono sempre coincidere

2. Quali affermazioni relative al context switch sono corrette?

1. Richiede di salvare lo stato del vecchio processo e caricare lo stato del nuovo processo
2. Non utilizza informazioni presenti nel PCB
3. È tempo utile di esecuzione del programma applicativo
4. Avviene quando la CPU passa dall'esecuzione di un processo a un altro

3. Quali file descriptor sono normalmente già aperti all'avvio di un processo Unix?

1. 1 e 2, cioè `STDOUT_FILENO` e `STDERR_FILENO`
2. 0, cioè `STDIN_FILENO`, per lo standard input
3. 3, cioè `STDLOG_FILENO`, per il log di sistema obbligatorio
4. 10, cioè il file descriptor riservato al kernel per ogni processo

4. Quali attività fanno parte dell'inizializzazione BIOS/UEFI durante il boot?

1. Avviare direttamente tutti i servizi utente configurati in `systemd`
2. Montare sempre il filesystem root definitivo prima di caricare il bootloader
3. Eseguire il Power-On Self-Test e inizializzare componenti hardware come CPU, RAM e controller di I/O
4. Scansionare i dispositivi di storage secondo l'ordine di boot configurato

5. Quali affermazioni sui segnali Unix/Linux sono corrette?

1. I segnali sono identificati da un numero e da un nome simbolico, ad esempio `SIGINT`
2. I segnali possono essere generati solo dalla tastiera e mai dal kernel o da altri processi
3. Un segnale è una notifica software asincrona consegnata a un processo
4. Un segnale è sempre usato per trasferire grandi quantità di dati tra processi

6. Quali affermazioni sulle prestazioni degli HDD sono corrette?

1. Il tempo medio di accesso dipende solo dalla dimensione della cache software del sistema operativo
2. Le letture casuali di piccoli blocchi possono essere molto più lente dei trasferimenti sequenziali
3. Il tempo medio di accesso include seek time e rotational latency
4. Un HDD ha normalmente la stessa latenza di accesso della DRAM

7. Quali affermazioni su EOF, SIGPIPE ed EPIPE nelle pipe sono corrette?

1. Scrivere su una pipe senza lettori restituisce sempre successo
2. EPIPE indica che il buffer della pipe è pieno ma ci sono ancora lettori attivi
3. Scrivere su una pipe senza lettori può generare SIGPIPE oppure restituire -1 con EPIPE se il segnale è gestito o ignorato
4. La lettura dalla pipe può restituire 0 se l'estremo di scrittura è chiuso e tutti i dati sono stati letti

8. Quali affermazioni descrivono correttamente il Virtual File System?

1. Il VFS obbliga tutte le applicazioni a conoscere il formato interno del file system usato
2. Il VFS impedisce l'uso di file system remoti
3. Il VFS separa le operazioni generiche sui file dai dettagli implementativi del singolo file system
4. Il VFS permette di usare la stessa interfaccia di system call per diversi tipi di file system

9. Quali affermazioni su linker, loader e librerie dinamiche sono corrette?

1. Le librerie dinamiche devono essere copiate integralmente dentro ogni eseguibile
2. Il loader produce il codice sorgente C a partire dal file eseguibile
3. Il loader porta il programma eseguibile in memoria per consentirne l'esecuzione
4. Il linker combina file oggetto e librerie per produrre un file eseguibile

10. Quali affermazioni descrivono correttamente il ruolo della memoria nell'esecuzione di un programma?

1. Un programma deve essere trasferito dal disco in memoria per poter essere eseguito
2. La memoria principale è sempre veloce quanto i registri della CPU
3. La CPU può accedere direttamente alla memoria principale e ai registri
4. La CPU può accedere direttamente a qualunque dato sul disco senza passare dalla memoria

Foglio delle risposte
Simulazione del 4 giugno 2026 – Compito A

Domanda	Tipo	1	2	3	4	Punti
1	M	☐	☐	☐	☐	
2	M	☐	☐	☐	☐	
3	M	☐	☐	☐	☐	
4	M	☐	☐	☐	☐	
5	M	☐	☐	☐	☐	
6	M	☐	☐	☐	☐	
7	M	☐	☐	☐	☐	
8	M	☐	☐	☐	☐	
9	M	☐	☐	☐	☐	
10	M	☐	☐	☐	☐	
C	L					
Shell	L					

Esercizio C

Viene fornito il file `template.c`, contenente la funzione:

```
int anomaly_score(const char *line);
```

La funzione riceve una riga di log e restituisce un punteggio numerico che rappresenta il livello di anomalia della riga.

Il programma ottenuto compilando `template.c` deve essere invocato come segue:

```
./programma <file_log> <N>
```

dove:

- `<file_log>` è il nome del file da analizzare (viene fornito `sample.log` a titolo di esempio);
- `<N>` è il numero di righe da leggere dal file.

L'esercizio è suddiviso in tre step incrementali.

Step 1: Lettura da file e pipe padre-figlio (valutazione: 6/10)

Modificare `template.c` in modo che implementi il seguente comportamento:

1. il padre apre il file ricevuto da linea di comando;
2. il padre legge le prime N righe del file;
3. il padre crea N processi figli tramite `fork()`;
4. il padre invia al figlio, tramite pipe, la riga da analizzare;
5. ogni figlio:
 - riceve la propria riga dalla pipe;
 - calcola il punteggio usando `anomaly_score()`;
 - stampa il risultato su `stdout`;
 - termina;
6. il padre attende la terminazione di tutti i figli.

Ogni figlio deve stampare una riga nel formato:

```
riga <indice>: pid=<pid> score=<score>
```

Esempio:

```
riga 0: pid=12341 score=5
riga 1: pid=12342 score=12
riga 2: pid=12343 score=0
riga 3: pid=12344 score=8
```

L'ordine delle righe stampate non è necessariamente deterministico, perché i figli eseguono in parallelo.

Il programma deve avere nome `log-pipe.c`

Il comportamento del programma è verificabile eseguendo lo script `validate-step-1.sh` (fornito).

Step 2: Memoria condivisa anonima (valutazione: 8/10)

Modificare il programma precedente in modo che i figli non stampino più direttamente il risultato.

Poiché tutti i processi sono imparentati, il padre può creare una memoria condivisa anonima. La memoria condivisa deve contenere una struttura simile alla seguente:

```
#define MAX_LINES 64
#define MAX_LINE_LENGTH 256

struct result {
    pid_t pid;
    int score;
    char line[MAX_LINE_LENGTH];
};

struct shared_data {
    int n;
    struct result results[MAX_LINES];
};
```

Il padre deve:

1. aprire il file ricevuto da linea di comando;
2. leggere le prime N righe;
3. creare e inizializzare la memoria condivisa anonima;
4. creare N processi figli;
5. inviare a ogni figlio una riga tramite pipe;
6. attendere la terminazione di tutti i figli;
7. leggere i risultati dalla memoria condivisa;
8. stampare i risultati in ordine di indice;
9. liberare le risorse utilizzate.

Ogni figlio *i* deve:

- ricevere la propria riga dalla pipe;
- calcolare il punteggio usando `anomaly_score()`;
- scrivere il risultato nella posizione `results[i]` della memoria condivisa;
- terminare.

Il padre deve stampare i risultati in ordine di indice:

```
riga 0: pid=12341 score=5
riga 1: pid=12342 score=12
riga 2: pid=12343 score=0
riga 3: pid=12344 score=8
```

In questo step l'output deve essere ordinato per indice di riga.

Il programma deve avere nome `log-shm.c`

Il comportamento del programma è verificabile eseguendo lo script `validate-step-2.sh` (fornito).

Step 3: Processo riepilogatore per il calcolo delle statistiche (10/10)

Estendere la soluzione dello Step 2 nel modo seguente.

Dopo aver atteso la terminazione di tutti i figli, il padre non stampa i risultati finali su `stdout`. Invece crea un nuovo processo figlio che si occuperà di stampare i risultati e alcune statistiche riepilogative.

Il nuovo processo figlio deve:

1. leggere tutti i risultati prodotti dai figli elaboratori dalla memoria condivisa;
2. stamparli su `stdout`;
3. calcolare e stampare:
 - numero di righe elaborate;
 - somma totale degli score;
 - score medio;
 - score massimo;
 - indice della riga con score massimo;
4. terminare.

Esempio:

```
riga 0: pid=12341 score=5
riga 1: pid=12342 score=12
riga 2: pid=12343 score=0
riga 3: pid=12344 score=8
```

```
righe elaborate: 4
score totale: 25
score medio: 6.25
score massimo: 12
riga con score massimo: 1
```

Il programma deve avere nome `log-shm-summary.c`

Il comportamento del programma è verificabile eseguendo lo script `validate-step-3.sh` (fornito).

Esercizio Shell

Descrizione del task

Dato un file di log del server web di grandi dimensioni `access.log` (es. Apache o Nginx) e un log di autenticazione (`auth.log`), gli studenti devono creare una pipeline forense.

Lo script deve:

- Rilevare potenziali attacchi brute-force (molti tentativi di login falliti dallo stesso IP)
- Identificare il blocco IP utilizzando i primi due byte dell'indirizzo IP (es. *155.185* è il *prefisso* dell'IP 155.185.48.199)
- Generare un report degli attori malevoli contenente i blocchi IP e il numero di fallimenti per ciascuno di essi
- Generare un report riassuntivo con i 10 indirizzi IP e i 10 prefissi che hanno generato il maggior numero di problemi

Le attività sospette possono essere riassunte come:

- Codici di risposta 400 (Bad request) in `access.log`
- Codici di risposta 401 (Authorization required) in `access.log`
- Password errata da ssh in `auth.log` (Le righe da cercare sono nella forma:
`<DATA> <HOSTNAME> sshd[<PID>]: Failed password for <USER> from <IP> port <PORT> ssh2)`

Lo script prodotto dagli studenti riceve in input due parametri, ovvero il file `access.log` e il file `auth.log`. Lo script deve essere convalidato dallo script `validator.sh` fornito. Il validatore e i relativi file di dati di supporto devono essere decompressi in una directory. Il validatore deve essere avviato da quella directory con il nome dello script come unico parametro.

Dettagli dei passaggi (e valutazione)

- **Step 1 (3/10):** Creare una funzione che elabori un file `access.log` ed estraiga gli IP delle richieste con problemi (codice di risposta 400 o 401) salvandoli in un file `web.ip` (un IP per riga, ogni IP può comparire più volte).
- **Step 2 (6/10):** Creare una funzione che operi sul file `auth.log` ed estraiga gli IP delle richieste di login fallite (`Failed password`) salvandoli in un file `ssh.ip` (un IP per riga, ogni IP può comparire più volte).
- **Step 3 (8/10):** Creare una funzione che unisca i file `web.ip` e `ssh.ip` contando per ogni IP il numero di problemi. L'output sarà un file `bad_ip.csv` con il formato `<IP>, <count>` (in questo file ogni IP può comparire una sola volta).
- **Step 4 (9/10):** Creare una funzione che aggregi le righe del file `bad_ip.csv` in base al prefisso di rete. L'output sarà un file `bad_prefix.csv` con il formato `<prefisso>, <count>` (ogni prefisso può comparire una sola volta).
- **Step 5 (10/10):** Creare una funzione di reportistica che restituisca i 10 peggiori IP e i 10 peggiori prefissi basandosi su `bad_ip.csv` e `bad_prefix.csv`.

Di seguito è riportato un esempio del formato del report:

```
*** 10 worst IP
155.185.1.1: 10
155.185.48.199: 5
...
*** 10 worst prefixes
155.185: 15
...
```